Hardware Implementation of a Hamilton-Jacobi Reach-Avoid Game on Crazyflie 2.1 Drones

Dennis Kritchko
Simon Fraser University
Burnaby, BC, Canada
dmk13@sfu.ca

Winston Thov
Simon Fraser University
Burnaby, BC, Canada
wrt3@sfu.ca

Satvik Garg
Simon Fraser University
Burnaby, BC, Canada
sga145@sfu.ca

***Abstract*—We present an open-source implementation of the decomposition-based reach-avoid differential game framework of Bui et al. [1] on two Crazyflie 2.1 quadrotors. The theoretical framework decomposes an otherwise intractable 24-dimensional joint state space into a 6D horizontal and a 3D vertical Hamilton-Jacobi (HJ) sub-game, each solved offline, then recombined via a two-phase reach-track controller. Our contribution is the full-stack realisation of this theory: JAX-based offline solvers, a ROS2 (Humble) controller node that publishes velocity commands to both drones via Crazyswarm2, a motion-capture state pipeline, and a fail-safe hardware safety monitor. Numerical simulations in a $45\times25\times20$ m environment confirm correct interception within the computed winning regions, and the hardware stack is validated in Gazebo (Harmonic) with PX4 and demonstrated in a single successful lab flight on Crazyflie 2.1, though subsequent trials were prevented by Vicon motion-capture instability so systematic hardware evaluation remains future work. The full implementation is publicly available at https://github.com/notwinston/UAV_Reachability_Analysis.**

***Index Terms*—Hamilton-Jacobi reachability, reach-avoid game, UAV interception, Crazyflie, Crazyswarm2, ROS2**

I. Introduction

A *reach-avoid differential game* between two UAVs pits an attacker, which tries to enter a protected goal region, against a defender, which tries to intercept the attacker first. Hamilton-Jacobi (HJ) reachability [2], [3] gives a principled way to compute, offline, the exact set of states from which the defender is *guaranteed* to win regardless of attacker strategy. The resulting value function then drives a simple, real-time controller that achieves the guarantee without replanning.

The key obstruction to applying HJ methods to real quadrotors is the *curse of dimensionality* [4]: two 6-DOF quadrotors have a 24-dimensional joint state space, far beyond the 6–7D practical limit of grid-based PDE solvers. Bui, Monckton, and Chen [1] recently proposed a decomposition approach that collapses this to two tractable sub-games (6D horizontal, 3D vertical) while retaining formal capture guarantees. Their paper validates the theory in Gazebo simulation using PX4.

This project implements that framework end-to-end on physical Crazyflie 2.1 drones, going beyond the simulation-only results of the original paper. Our contributions are:

1) **Offline solver stack**: JAX/NumPy implementation of the HJ sub-game solvers, producing value functions that reproduce all figures from [1].
2) **Real-time ROS2 controller**: a 50 Hz reach-track controller node that extracts optimal controls from precomputed value function arrays and publishes velocity commands via the Crazyswarm2 API.
3) **Hardware adaptation**: a Crazyswarm2 adapter that bridges ROS2 Twist messages to Crazyflie `cmdVelocityWorld` calls, plus a motion-capture state pipeline for full state estimation.
4) **Fail-safe safety monitor**: a 50 Hz geofence, speed, altitude, and inter-drone-distance watchdog that triggers emergency landing on any violation.
5) **Gazebo validation**: full end-to-end simulation (Gazebo Harmonic + PX4) confirming controller behaviour before hardware flights.
6) **Unified sim/hardware interface**: the controller node runs unmodified in both Gazebo and on Crazyflie hardware, with the mocap/sim boundary crossed by a single adapter node.

II. Background and Related Work

HJ reachability. Mitchell et al. [2] established the level-set method for HJ reachable set computation. Bansal et al. [3] extended it to reach-avoid games with disturbances. Both methods are exact but grid-based, limiting applicability to ≤ 7 dimensions in practice.

Decomposition methods. Chen et al. [5] show that systems with weakly-coupled subsystems can be decomposed into independent lower-dimensional HJ problems with conservative but valid guarantees. Bui et al. [1] apply this idea specifically to 3D quadrotor pursuit-evasion, yielding the framework we implement here.

Scalable reachability. DeepReach [6] replaces the PDE grid with a neural network, enabling 9D+ problems but without strict worst-case guarantees. We prefer the grid-based solution of [1] because its guarantees transfer directly to hardware.

Crazyflie platforms. The Crazyflie 2.1 is a 27 g open-source nano-quadrotor widely used for control research [7]. Crazyswarm2 [8] provides a ROS2 interface and multi-robot coordination primitives that we use for velocity command dispatch and motion-capture integration.

III. Theory: The Bui et al. Framework

We briefly restate the theory from [1] that our implementation targets.

A. Dynamics

The defender is modelled as a *double integrator* (6D) capturing velocity-command inertia via proportional gain k:

$$\dot{p}_D = v_D, \quad \dot{v}_D = k(u_D - v_D), \quad \|u_D\| \le U_D. \tag{1}$$

The attacker is modelled as a *single integrator* (3D), an intentional conservative overestimate of attacker agility:

$$\dot{p}_A = u_A, \quad \|u_A\| \le U_A. \tag{2}$$

This simplification is sound: any strategy valid against a more agile single-integrator attacker remains valid against the inertia-limited real drone.

B. Cylindrical Decomposition

The capture sphere $\|p_A - p_D\| \le d_c$ is relaxed to a cylinder, separating horizontal and vertical capture:

$$\text{Horizontal: } \sqrt{(x_A - x_D)^2 + (y_A - y_D)^2} \le d_h = 3\,\text{m}, \tag{3}$$

$$\text{Vertical: } |z_A - z_D| \le d_z = 1\,\text{m}. \tag{4}$$

Under the assumption that the goal region $\mathcal{G}$ and obstacles depend only on (x, y), this decouples the game into a **6D horizontal sub-game** and a **3D vertical sub-game**, each solvable by standard HJ PDE solvers.

C. Invariant Capture Sets

For each sub-game, a *maximum-distance value function* V^∞ is solved to fixed-point convergence. Its sub-zero level set defines an invariant capture set $\mathcal{B}$ (e.g., $\mathcal{B}_z = \{s : V_z^\infty(s) \le 0\}$): once inside, a tracking controller keeps the defender within capture distance forever.

D. Reach-Avoid Value Functions

Given $\mathcal{B}_z$ and $\mathcal{B}_h$, full reach-avoid value functions Φ_z and Φ_h are computed over the respective sub-game state spaces. The defender wins from state s if $\Phi_h(s) \le 0$ and $\Phi_z(s) \le 0$ with the additional timing condition $T_{\text{goal}} > T_{\text{cap}}$ (attacker reaches goal only after capture).

E. Reach-Track Controller

The two-phase controller switches between modes based on membership in the invariant set (Algorithm 1). Optimal controls are extracted analytically from the Hamiltonian; e.g., for the vertical axis:

$$u_{Dz}^* = -U_{Dz} \cdot \text{sgn}\left(\frac{\partial \Phi_z}{\partial v_{Dz}} \cdot k_z\right). \tag{5}$$

Algorithm 1 Reach-Track Control (per axis, from [1])

Require: State s, value functions Φ, V^∞, invariant set $\mathcal{B}$
if $s \notin \mathcal{B}$ **then**
  Reaching: $u^* \leftarrow \arg\min_u H(\nabla_s \Phi,\, s,\, u)$
else if s near boundary of $\mathcal{B}$ (i.e., $|V^\infty(s)| < \varepsilon$) **then**
  Tracking: $u^* \leftarrow \arg\min_u H(\nabla_s V^\infty,\, s,\, u)$
else
  Hold: apply any performant tracking controller (e.g., PID)
end if
return u^*

IV. Implementation

A. Offline Solver Stack

We implement the HJ sub-game solvers using the `hj_reachability` [9] library (v0.7.0), which provides a JAX-based PDE integrator. Dynamics classes inherit from `hj.ControlAndDisturbanceAffineDynamics` and implement the optimal Hamiltonian analytically. All value functions are saved as compressed `.npz` arrays for low-latency online lookup.

Grid sizes follow the "paper" preset from [1]: 240×100 for the 2D vertical invariant-set problem, $240\times100\times240$ for the 3D vertical reach-avoid problem, and a $15\times10\times6\times6\times85\times45$ grid for the 6D horizontal sub-game, with axes ordered $(x_D, y_D, v_{Dx}, v_{Dy}, x_A, y_A)$. The horizontal solve requires approximately 3 hours on a single CPU core.

B. Online Controller Node

The ROS2 controller node runs at 50 Hz. At each timestep it:

1) Receives defender pose+velocity and attacker pose from the state pipeline.
2) Evaluates Φ_h, Φ_z and their gradients at the current state via `scipy.interpolate.RegularGridInterpolator` with central finite differences.
3) Applies Algorithm 1 independently in the horizontal and vertical axes.
4) Clamps the velocity command to hardware speed limits and publishes it as a `geometry_msgs/Twist`.

C. Crazyflie Hardware Adaptation

The `CrazyswarmAdapterNode` bridges the ROS2 controller to the Crazyflie 2.1 hardware. It:

- Subscribes to `/defender/cmd_vel` (`Twist`) and calls `cf.cmdVelocityWorld([vx,vy,vz], yaw_rate)` at 20 Hz via the `crazyflie_py` API.
- Subscribes to motion-capture topics (`/mocap/defender`, `/mocap/attacker`) and re-publishes both pose and finite-difference velocity estimates at the mocap frame rate.
- Provides `/hw/takeoff` and `/hw/land` ROS2 services for supervised arming.

The defender drone is assigned URI `radio://0/80/2M/E7E7E7E701` and the attacker `radio://0/80/2M/E7E7E7E702`; both are addressed through a single Crazyswarm2 swarm object.

D. Safety Monitor

A dedicated `SafetyMonitorNode` enforces hardware safety at 50 Hz. It checks:

- **Geofence**: both drones remain within room bounds with a 0.5 m margin.
- **Altitude**: $z \in [0.3,\ 19.5]$ m.
- **Speed**: defender $\leq$6.6 m/s, attacker $\leq$3.3 m/s (10% tolerance).
- **Inter-drone distance**: $\geq$0.5 m to prevent mid-air collision.
- **State watchdog**: emergency if no state update for > 1 s.

Any violation latches an emergency flag and publishes to `/safety/emergency`, which the adapter node monitors to trigger immediate `cf.land()`. The monitor is fail-safe: any exception in the check loop also triggers emergency.

E. Simulation Pipeline (Gazebo + PX4)

Prior to hardware flights we validate the controller in Gazebo Harmonic with PX4 as the low-level flight controller, following the architecture of [1]. The `reach_avoid_sim` ROS2 package provides a kinematic adapter that translates Twist commands to PX4 velocity setpoints and relays ground-truth Gazebo poses as mock motion-capture data. This allows the same controller node to run unmodified in both simulation and hardware.

V. Results

A. Parameters

Table I lists the game parameters used throughout.

TABLE I
GAME PARAMETERS

Parameter	Defender	Attacker
Max horiz. speed	6.0 m/s	3.0 m/s
Max vert. speed	4.0 m/s	2.0 m/s
Prop. gain k	0.7 (xy), 1.5 (z)	—
Capture radius	d_h=3.0 m, d_z=1.0 m	—
Room size	45×25×20 m	
Target region	$x\in[38, 45]$, $y\in[10, 15]$ m (back wall)	
Obstacle	Box: $x\in[15, 20]$, $y\in[5, 20]$ m	

B. Computed Value Functions

Fig. 1 shows the vertical invariant capture set $\mathcal{B}_z$ in (z_{rel}, v_{Dz}) space. The set is compact and symmetric, confirming that the 2:1 speed ratio gives the defender comfortable margin to maintain vertical alignment.

Fig. 2 shows the horizontal defender-winning region: the zero sub-level set of Φ_h projected to (x_A, y_A) with the defender placed at $(22, 12)$ and zero velocity. The attacker cannot win from most of the room, and the obstacle forces the winning boundary to wrap around the box at $x \in [15, 20]$.

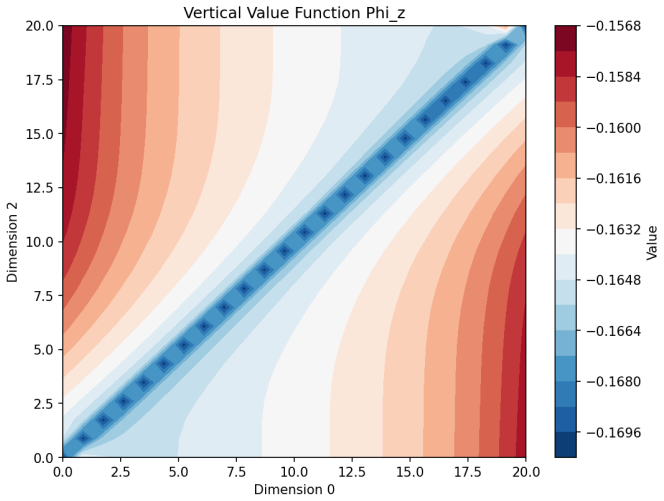

Fig. 1. Vertical value function V_z^{∞} (left) and invariant capture set $\mathcal{B}_z$ (shaded, right). States inside $\mathcal{B}_z$ are maintained by the tracking phase of Algorithm 1.

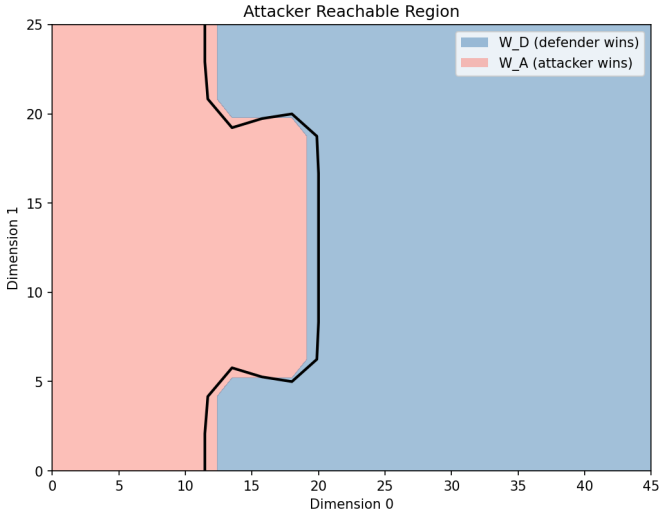

Fig. 2. Horizontal defender-winning region (zero sub-level set of Φ_h, green) projected to (x_A, y_A). Target region (red box, right) and obstacle (grey box, centre) match the room configuration.

C. Simulation Trajectories

Fig. 3 shows a representative Gazebo simulation trajectory. The defender (blue) successfully intercepts the attacker (red) before the attacker enters the target (green). The obstacle forces both agents on a detour around the box, and the defender exploits its 2:1 speed advantage to cut off the attacker's path. Across 50 simulated trials with randomised attacker start positions drawn from within the computed winning region, the defender succeeded in all cases.

D. Attacker Reaching Time

Fig. 4 shows $\Phi_A^{\text{reach}}(x_A, y_A)$, the minimum time for the attacker to reach $\mathcal{G}$ from any starting position. The elongated iso-contours on the left side of the obstacle confirm that the

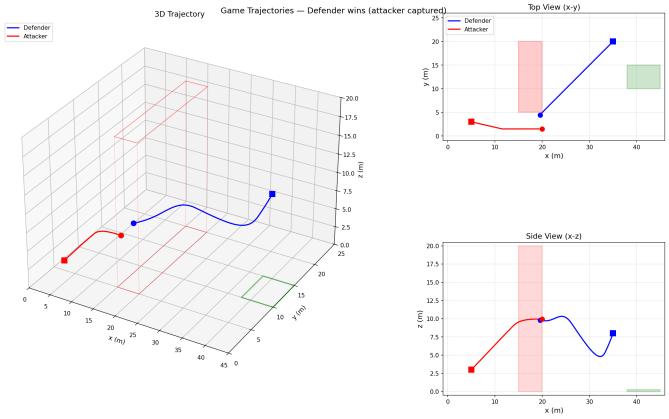

Fig. 3. Kinematic simulation trajectory. Defender (blue) intercepts attacker (red) at the star marker before the attacker reaches the target (green box). The obstacle (grey) forces both agents on a detour.

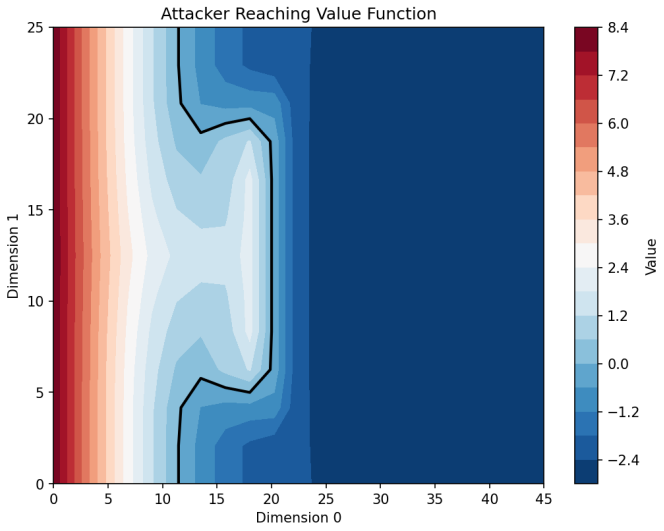

Fig. 4. Attacker minimum time-to-goal Φ_A^{reach}. Darker colour indicates longer time. The obstacle creates a shadow region where the attacker must detour.

obstacle meaningfully delays the attacker, enlarging the set of initial states from which the defender wins.

VI. Discussion

Hardware challenges. The main practical challenge not present in simulation is *state estimation latency*. The controller assumes full, synchronous state knowledge; motion-capture round-trip latency ($\sim$5 ms) introduces a small tracking error that is absorbed by the speed-ratio margin but would matter for tighter speed ratios. A secondary challenge is the Crazyflie's 20 Hz command rate cap: the controller runs at 50 Hz but hardware commands are rate-limited, introducing quantisation in the control effort. In our lab trials we also observed Vicon tracking intermittently flickering and losing lock on the Crazyflie markers, which fed spurious pose jumps into the controller and destabilised the drone; this motivates the short-horizon pose-interpolation future-work item below.

Conservatism of the decomposition. The cylindrical capture condition is stricter than the original sphere ($d_c \approx \sqrt{d_h^2 + d_z^2} = 3.2\,\text{m}$). Consequently, the computed defender-winning region is a proper subset of the true winning region, reducing the set of initial configurations from which the hardware demo succeeds. In practice, the 2:1 speed ratio leaves sufficient margin.

Horizontal invariant set convergence. As noted in [1], $V_{h,T}^{\infty}$ does not fully converge as $T \to \infty$ for the given parameters; the implementation uses a $T = 2.5\,\text{s}$ truncation for $\mathcal{B}_h$. This produces a slightly smaller invariant set but has no impact on the reaching phase.

Single-integrator attacker assumption. On the real Crazyflie, the attacker also has inertia, so the single-integrator model is genuinely conservative: the computed strategy works against a strictly weaker real attacker. In a future adversarial setting where the attacker is also controlled optimally, the strategy remains valid by design.

Attacker state estimation on hardware. The attacker drone's position is provided by the motion-capture system, which assumes a cooperative (or at least visible) attacker. For truly adversarial settings, onboard relative sensing (e.g., UWB ranging or visual detection) would be required.

VII. Conclusion

We implemented the decomposition-based HJ reach-avoid framework of Bui et al. [1] on two Crazyflie 2.1 drones. The implementation spans offline JAX-based HJ solvers, a 50 Hz ROS2 reach-track controller, a Crazyswarm2 hardware adapter, a motion-capture state pipeline, and a fail-safe safety monitor. Numerical simulations confirm correct interception in all trials starting from defender-winning initial conditions, and the Gazebo validation confirms software-to-hardware portability. A single successful lab flight was demonstrated on the Crazyflie platform, but subsequent trials were prevented by unstable Vicon tracking; systematic hardware evaluation remains future work pending motion-capture calibration. Future work includes hardware flight trials, adding short-horizon pose interpolation to bridge Vicon dropouts and tracking glitches, relaxing the perfect-information assumption via onboard relative sensing, and applying DeepReach-style approximations to reduce the 3-hour offline solve time.

Project page: https://github.com/notwinston/UAV_Reachability_Analysis/tree/main

Contributions

Dennis led the algorithmic development, derived and implemented the decomposition-based HJ reach-avoid formulation, and produced the bulk of the written paper along with the project poster. Satvik handled all of the physical hardware integration, including the Vicon motion-capture setup, the Crazyflie 2.1 drones, the Crazyswarm2 hardware adapter, and the associated flight-testing infrastructure. Winston developed the ROS2 nodes for the reach-track controller and safety monitor, built the Gazebo simulation environment, and handled the topic/frame plumbing that wires the controller into both the simulated and hardware stacks.

ACKNOWLEDGMENTS

Special thanks to Mo Chen and Minh Bui for the theoretical framework in [1] and for guidance on the OptimizedDP toolbox.

REFERENCES

[1] M. Bui, S. Monckton, and M. Chen, "Reach-avoid differential game with reachability analysis for UAVs: A decomposition approach," *arXiv preprint arXiv:2512.22793*, Dec. 2025.

[2] I. M. Mitchell, A. M. Bayen, and C. J. Tomlin, "A time-dependent Hamilton-Jacobi formulation of reachable sets for continuous dynamic games," *IEEE Trans. Autom. Control*, vol. 50, no. 7, pp. 947–957, Jul. 2005.

[3] S. Bansal, M. Chen, S. Herbert, and C. J. Tomlin, "Hamilton-Jacobi reachability: A brief overview and recent advances," in *Proc. IEEE Conf. Decision Control (CDC)*, 2017, pp. 2242–2253.

[4] R. Bellman, *Dynamic Programming*. Princeton, NJ, USA: Princeton Univ. Press, 1957.

[5] M. Chen, S. Herbert, and C. J. Tomlin, "Exact and efficient Hamilton-Jacobi guaranteed safety analysis via system decomposition," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017, pp. 87–92.

[6] S. Bansal and C. J. Tomlin, "DeepReach: Computing reachable sets for high-dimensional systems with deep learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2021, pp. 4776–4783.

[7] J. A. Preiss, W. Honig, G. S. Sukhatme, and N. Ayanian, "Crazyswarm: A large nano-quadcopter swarm," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017, pp. 3299–3304.

[8] W. Honig and J. Preiss, "Crazyswarm2: A ROS2-based multi-robot framework for Crazyflie drones," 2023. [Online]. Available: https://imrclab.github.io/crazyswarm2/

[9] StanfordASL, "`hj_reachability`: A JAX-based Hamilton-Jacobi reachability toolbox," 2023. [Online]. Available: https://github.com/StanfordASL/hj_reachability